



EAST WEST UNIVERSITY

Project Report

Topic: “University Management System”

Course Code: CSE 423

Course Title: Software Architecture

Semester: Spring 2026

Section: 01

Submitted by

Tabassum Nahar Yeah

2023-1-60-068

Submitted to

Ahmed Adnan

Lecturer

Department of Computer Science and Engineering

Date: 1st May, 2026

Transport Management System of UMS

Applying SOLID Principle

Previous Code and output:

```
import java.awt.*;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.net.URISyntaxException;
import java.util.Random;
import java.util.Scanner;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.net.URI;

//Shahzaneer Ahmed --> SP21-BCS-087
// input fields are validated (exceptional handling)
// randomly generation Bus Registration ID
//CRUD
// file handling

public class TMS {

    public static void manage (String regNumber) {
        System.out.print("\n");
        Scanner input = new Scanner(System.in);

        String registrationNo = regNumber;
        String busID = busID(registrationNo);

        boolean isRegistered = busFileFound(regNumber);
        boolean feePaid = false;

        String busReg = "";
        while (true) {
            System.out.println("Enter Corresponding Number to Select the option");
            System.out.println("1. Bus Routes\n2. Bus fees\n3. Bus registration\n4. Update Info\n5. fee " +
                "Payment\n6. Bus Profile\n7. Delete Bus registration\n8. Previous Menu\n9. Exit TMS\n");
            String selection = input.next();
```

```

        if (selection.equals("9")) {
            System.out.println("Thank you for using Transport Management
System !");
            System.exit(0);
        }
        else if (selection.equals("8")) {
            TransportManagement.manage(regNumber);
        }
        else if
(selection.equals("1")||selection.equals("2")||selection.equals("3")||selection
.equals("4")||selection.equals("5")||selection.equals("6")||selection.equals("7
")){
            switch (selection) {
                case "1" ->{
                    displayRoutesAndStops(registrationNo);
                    try {
                        Thread.sleep(4000);
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
                case "2" ->{
                    displayBusFees(registrationNo);
                    try {
                        Thread.sleep(4000);
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
                case "3" ->{

                    if(busFileFound(registrationNo)){
                        System.out.println("Your Account Already exists in
our database !");

                        isRegistered = true;

                    }

                    else if (!busFileFound(registrationNo)){
                        busReg = busRegistration(registrationNo);
                        //      text file generation with the busID number
                        File busFile = new
File("transport/TransportFiles/"+registrationNo+".txt");
                        try {
                            busFile.createNewFile();
                            isRegistered = true;
                        }
                        catch (IOException e) {

```

```

        System.out.println("Sorry we found some issues
creating your file !");
    }
//    writing details in text file
    try {
        FileWriter busFileWriter = new
FileWriter("transport/TransportFiles/"+registrationNo+".txt");
        busFileWriter.write(busReg);
        System.out.print("\n*****\n\n");
Registered Successfully! *****\n\n");
        busFileWriter.close();
    }
    catch (IOException e) {
        System.out.println("Sorry we found some issues
inserting your details in the our database ");
    }
}

case "4" ->{
//    update your registered data!
    if(isRegistered){
        busReg = busRegistration(registrationNo);
        feePaid = false;
        try {
            FileWriter busFileWriter = new
FileWriter("transport/TransportFiles/"+registrationNo+".txt");

            busFileWriter.write(busReg);
            System.out.print("\n***** Data
Updated Successfully! " +
                "*****\n\n");
            busFileWriter.close();
        }
        catch (IOException e) {
            System.out.println("Sorry we found some issues
inserting your details in the our database ");
        }
    }
    else{
        System.out.println("Kindly Register first !");
    }
}

case "5" ->{
    if(!feePaid){
        paymentMethod(registrationNo,isRegistered);
        feePaid = true;
    }
}

```

```

    }
    else if(feePaid) System.out.println("You have already
paid the fee !");

    }

    case "6" -> {
        if (!feedback && isRegistered) {
            String busReview =
busReviewMessage(registrationNo);
            feedback = true;
            adding this review to the file
            try {
                FileWriter busFileWriter = new
FileWriter("E:\\transport\\TransportFiles\\" + registrationNo + ".txt",
                true);
                String busReviewFinal =
String.format("\n\nYour FeedBack --->\n %s", busReview);
                busFileWriter.write(busReviewFinal);
                busFileWriter.close();
            } catch (IOException e) {
                System.out.println("Sorry we found some
issues inserting your details in the our database ");
            }

        }

        } else if (!isRegistered) System.out.println("You
cannot give feedback without registering to TMS ");
        else if (busFileFound(registrationNo))
System.out.println("You have already given the " +
"feedback !");

    }

    case "6" ->{
        bus profile --
        File busFilex = new
File("transport/TransportFiles/"+registrationNo+".txt");

        if (!busFilex.exists()) System.out.println("Your Profile
doesn't exist!");

        else if(feePaid && busFilex.exists()){
            try {
                File busFile = new
File("transport/TransportFiles/"+registrationNo+".txt");
                Scanner sc = new Scanner(busFile);

```

```

        while (sc.hasNextLine()) {
            String profile = sc.nextLine();
            System.out.println(profile);
        }
        sc.close();
    }

    catch(IOException e){
        System.out.println("We found some issues in
reading your data! try again!");
    }
}

else if (!isRegistered) System.out.println("Your profile
doesn't exist! Register first!");
else System.out.println("Kindly pay the fee to proceed
to your Profile !");

}

case "7" ->{
    File busFile = new
File("transport/TransportFiles/"+registrationNo+".txt");
    if (busFile.delete()) {
        System.out.println("The bus profile has been deleted
successfully!");

        isRegistered = false;
        feePaid = false;
    }
    else if (!busFile.exists()) System.out.println("No
record found! you haven't registered yet!");
    else System.out.println("Something went wrong");

}

}

} else System.out.println("invalid input!");

}

}

// Methods for TMS

// displaying routes, stops and fees
static void displayBusFees(String registrationNo){
    System.out.println("***** Buses Fees *****");
}

```

```

        System.out.println("Route 1 : 20,000\nRoute 2 : 15,000\nRoute 3 :
14,000\nRoute 4 : 13,000\nRoute 5 : 10,000\nRoute 6 : 22,000\nRoute 7 :
24,000\nRoute 8 : 21,000\n\n");

    }
    static void displayRoutesAndStops(String registrationNo){
        System.out.println("-----Route #
1-----");
        System.out.println("Stops -->\n F-11 Markaz\nGolra more\nMargala
road\nPAF Complex Gate\nShaheen " +
            "Chowk\nMadina" +
            " Market");
        System.out.println("-----Route #
2-----");
        System.out.println("Stops -->\nG-11/2 Apple School\n G-11 Markaz \n
G-11/3 Alladin Appartment\n G-10 Umer " +
            "market\n G-9/1 Street no 44");
        System.out.println("-----Route #
3-----");
        System.out.println("Stops --> \nAskari XI\n Qasim Market \n Roomi Market
\n GPO Saddar\n Kacheri Chowk");
        System.out.println("-----Route #
4-----");
        System.out.println("Stops -->\n Hajj Complex \n Old Daewoo Stop \n
Pirdhadhai More \n Charing Cross \n Valley" +
            " Road");
        System.out.println("-----Route #
5-----");
        System.out.println("Stops -->\nChungi no 22 \n CMH \n 502 workshop \n
Jinnah Park \n Jhandha Chechi");
        System.out.println("-----Route #
6-----");
        System.out.println("Stops -->\nBahria Town Ace Academy\n Street no 6
Bahria More\n Al Noor Pharmacy \nRahat " +
            "Bakery\n PWD Barrier \n London Tradex");
        System.out.println("-----Route #
7-----");
        System.out.println("Stops -->\nGangal East\n karal Chowk\nGangal
West\nFizaya Colony\n Chatry Chowk\n Khanna " +
            "Pull");
        System.out.println("-----Route #
8-----");
        System.out.println("Stops -->\nNawal Anchorage\n Bhander \n Sowam
Garden\n Al Shifa Hospital\n Toyota " +
            "Motors\n\n");

    }

```

```

//    bus registration
static String busID(String registrationNo){
    //    busID generation
    Random randCardNum = new Random();
    int number =(int) 1000 + randCardNum.nextInt(1000);
    String busID = "" + number;
    return busID;
}

static String registrationTiming(String registrationNo){
    //    Registration date and timing
    return
LocalDateTime.now().format(DateTimeFormatter.ofPattern("dd-MM-yyyy || h:m a"));

}

//    bus route validation
static String busRoute(String registrationNo){
    Scanner input = new Scanner(System.in);
    String route;
    while(true){

        System.out.println("Enter your route :");
        route = input.next();
        if (route.equals("1") || route.equals("2")|| route.equals("3") ||
route.equals("4") || route.equals(
            "5") || route.equals("6") || route.equals("7") ||
route.equals("8")){
            return route;
        }
        else System.out.println("Enter valid route no!");
    }

}

static boolean find(String routeName, String [] arr){
    for (int i = 0; i < arr.length;i++){
        if (routeName.equalsIgnoreCase(arr[i])){
            return true;
        }
    }
    return false;
}

static String [] routesNameStr(String registrationNo, String route){
    String [] route_1 = {"F-11 Markaz","Margala road","Golra More","PAF
Complex Gate","Shaheen Chowk","Madina Market"};
    String [] route_2 = {"G-11/2 Apple School","G-11 Markaz","G-11/3 Alladin
Market","G-10 Umer Market","G-9/1 Street no 44"};
    String [] route_3 = {"Askari XI","Qasim Market","Roomi Market","GPO
Saddar","Kacheri Chowk"};

```

```

        String [] route_4 = {"Hajj Complex","Old Daewoo Stop","Pirdhadhai
More","Charing Cross","Valley Road"};
        String [] route_5 = {"Chungi no 22","CMH","502 workshop","Jinnah
Park","Jhandha Chechi"};
        String [] route_6 = {"Bahria Town Ace Academy","Street no 6 Bahria
More","Al Noor Pharmacy","Rahat Bakery",
        "PWD Barrier","London Tradex"};
        String [] route_7 = {"Gangal East","karal Chowk","Gangal West","Fizaya
Colony","Chatry Chowk","Khanna Pull"};
        String [] route_8 = {"Nawal Anchorage","Bhander","Sowan Garden","Al
Shifa Hospital","Toyota Motors"};

        switch (route){
            case "1" -> {
                return route_1;
            }
            case "2" -> {
                return route_2;
            }
            case "3" -> {
                return route_3;
            }
            case "4" -> {
                return route_4;
            }
            case "5" -> {
                return route_5;
            }
            case "6" -> {
                return route_6;
            }
            case "7" -> {
                return route_7;
            }
            case "8" -> {
                return route_8;
            }
        }
        return new String[]{};
    }
    static String busStop(String registrationNo, String route){
        Scanner input = new Scanner(System.in);
        String routeName;
        while(true) {
            System.out.println("Enter the stop name :");
            routeName = input.nextLine();
            if (find(routeName,routesNameStr(registrationNo,route))){
                break;
            }
        }
    }

```

```

        else System.out.println("Invalid route Name!");

    }
    return routeName;
}

//      bus fee
static String busFee(String registrationNo ,String route){
    String fees = "";
    switch (route) {
        case "1" -> fees = "20000";
        case "2" -> fees = "15000";
        case "3" -> fees = "14000";
        case "4" -> fees = "13000";
        case "5" -> fees = "10000";
        case "6" -> fees = "22000";
        case "7" -> fees = "24000";
        case "8" -> fees = "21000";
    }
    return fees;
}

//      bus timing selection

static String busDropOff(String registrationNo){
    while(true){
        Scanner input = new Scanner(System.in);
        //      selecting the drop-off timing

        System.out.println("Enter your preferred timing for drop-off\n1.
1:30 PM\n2. 2:30 PM\n3. 5:30 PM");
        String selectionDropOff = input.next();
        String dropOffTime = "";
        if (selectionDropOff.equals("1") || selectionDropOff.equals("2") ||
selectionDropOff.equals("3")){
            switch (selectionDropOff){
                case "1" -> dropOffTime = "1:30 PM";
                case "2" -> dropOffTime = "2:30 PM";
                case "3" -> dropOffTime = "5:30 PM";
            }
            return dropOffTime;
        }
        else System.out.println("kindly Enter valid input!");
    }

}

}

static String busPickUp(String registrationNo){

```

```

        Scanner input = new Scanner(System.in);
        while (true){
//            selecting the pick-up timing
            System.out.println("Enter your preferred timing for pick-up\n1. 8
AM\n2. 10 AM\n3. 12 AM");
            String selectionPickUp = input.next();
            String pickUpTime = "";
            if (selectionPickUp.equals("1") || selectionPickUp.equals("2") ||
selectionPickUp.equals("3")){
                switch (selectionPickUp){
                    case "1" -> pickUpTime = "8 AM";
                    case "2" -> pickUpTime = "10 AM";
                    case "3" -> pickUpTime = "12 AM";
                }
                return pickUpTime;
            }
            else System.out.println("kindly Enter valid input!");
        }
    }

//    bus Review
    static String busReviewMessage(String registrationNo){

        Scanner input = new Scanner(System.in);
        System.out.println("Enter your Review for Transport Management ");
        String review = input.nextLine();
        System.out.println("Thank you for your feedback about Our Bus Management
System !");
        return review;
    }

    static void busReview(String registrationNo , boolean isRegistered){

    }

//bus registration main

    static String busRegistration(String registrationNo){
        Scanner input = new Scanner(System.in);
        String busID = busID(registrationNo);
        String dateAndTime = registrationTiming(registrationNo);
        String route = busRoute(registrationNo);
        String busStop = busStop(registrationNo,route);

        String busFee = busFee(registrationNo,route);
        String pickUp = busPickUp(registrationNo);
        String dropOff = busDropOff(registrationNo);

```

```

//      All details
String details_x = String.format("""
    Registration No          : %s
    Bus ID                   : %s
    Route                    : %s
    Stop Name                : %s
    Fee                      : %s
    Pickup Timing            : %s
    DropOff Timing           : %s
    Registered at            : %s

""", registrationNo, busID, route, busStop, busFee, pickUp, dropOff, dateAndTime);

    return details_x;

}

//  payment Method

static void paymentMethod(String registrationNo , boolean isRegistered) {

    if (isRegistered) {
        Desktop desk = Desktop.getDesktop();
        Scanner input = new Scanner(System.in);

        while (true) {
            System.out.println("Select your payment method :\n1.
JazzCash\n2. HBL\n3. Bank Alfalah");
            try {
                int option = input.nextInt();
                if (option >= 1 && option <= 3) {
                    switch (option) {

                        case 1 -> {
                            try {
                                desk.browse(new
URI("https://www.jazzcash.com.pk"));
                                System.out.println("Thank you for using
JazzCash for paying fee");

                            } catch (IOException | URISyntaxException e) {
                                System.out.println("Sorry we found an issue
try again!");
                            }
                        }

                        case 2 -> {
                            try {

```

```

                                desk.browse(new
URI("https://www.hbllibank.com.pk"));
                                System.out.println("Thank you for using HBL
for paying fee");

                                } catch (IOException | URISyntaxException e) {
                                    System.out.println("Sorry we found an issue
try again!");
                                }
                            }
                        case 3 -> {
                            try {
                                desk.browse(new
URI("https://netbanking.bankalfalah.com/"));
                                System.out.println("Thank you for using Bank
Alfalah for paying fee");

                                } catch (IOException | URISyntaxException e) {
                                    System.out.println("Sorry we found an issue
try again!");
                                }
                            }
                        }

                    }

                    break;
                } else System.out.println("Enter appropriate option !");
            } catch (Exception e) {
                System.out.println("Invalid input !");
                input.nextLine(); //to clear the buffer
            }
        }

        } else System.out.println("you have to register first to proceed
Payment!");
    }

//    if bus file found!
    static boolean busFileFound(String registrationNo){
        File busFile = new
File("transport/TransportFiles/"+registrationNo+".txt");
        if (busFile.exists()) return true;
    }

```

```
    else return false;  
}
```

Previous Output:

Transport Management System

1. Visit transport management system
2. Previous Menu
3. Exit

Enter your choice :

1

Enter Corresponding Number to Select the option :

1. Bus Routes
2. Bus fees
3. Bus registration
4. Update Info
5. fee Payment
6. Bus Profile
7. Delete Bus registration
8. Previous Menu
9. Exit TMS

1

-----Route # 1-----

Stops -->

F-11 Markaz
Golra more
Margala road
PAF Complex Gate
Shaheen Chowk
Madina Market

-----Route # 2-----

Stops -->

G-11/2 Apple School
G-11 Markaz
G-11/3 Alladin Appartment
G-10 Umer market
G-9/1 Street no 44

-----Route # 3-----

Stops -->

Askari XI
Qasim Market
Roomi Market
GPO Saddar
Kacheri Chowk

```
-----Route # 4-----
Stops -->
Hajj Complex
Old Daewoo Stop
Pirdhadhai More
Charing Cross
Valley Road
-----Route # 5-----
Stops -->
Chungi no 22
CMH
502 workshop
Jinnah Park
Jhandha Chechi
-----Route # 6-----
Stops -->
Bahria Town Ace Academy
Street no 6 Bahria More
Al Noor Pharmacy
Rahat Bakery
PWD Barrier
London Tradex
```

```
-----Route # 7-----
Stops -->
Gangal East
karal Chowk
Gangal West
Fizaya Colony
Chatry Chowk
Khanna Pull
-----Route # 8-----
Stops -->
Nawal Anchorage
Bhander
Sowan Garden
Al Shifa Hospital
Toyota Motors
```

```
2
***** Buses Fees *****
Route 1 : 20,000
Route 2 : 15,000
Route 3 : 14,000
Route 4 : 13,000
Route 5 : 10,000
Route 6 : 22,000
Route 7 : 24,000
Route 8 : 21,000
```

```
Enter the stop name :
Kacheri Chowk
Enter your preferred timing for pick-up
1. 8 AM
2. 10 AM
3. 12 AM
2
Enter your preferred timing for drop-off
1. 1:30 PM
2. 2:30 PM
3. 5:30 PM
2
***** Registered Successfully! *****
```

```
4
Enter your route :
3
Enter the stop name :
Kacheri Chowk
Enter your preferred timing for pick-up
1. 8 AM
2. 10 AM
3. 12 AM
2
Enter your preferred timing for drop-off
1. 1:30 PM
2. 2:30 PM
3. 5:30 PM
2
***** Data Updated Successfully! *****
```

```
5
Select your payment method :
1. JazzCash
2. HBL
3. Bank Alfalah
3
Thank you for using Bank Alfalah for paying fee
```

```
6
Registration No      : SP21-BCS-108
Bus ID               : 1758
Route                : 3
Stop Name            : Kacheri Chowk
Fee                  : 14000
PickUp Timing        : 10 AM
DropOff Timing       : 2:30 PM
Registered at        : 11-02-2026 || 12:18 AM
```

Updated Code and Output:

TMS:

```
import java.util.Scanner;
```

```
public class TMS {

    private RouteService routeService = new RouteService();
    private FileService fileService = new FileService();
    private PaymentService paymentService = new PaymentService();
    private BusService busService = new BusService(routeService);

    public void manage(String regNo) {

        Scanner input = new Scanner(System.in);
        boolean feePaid = false;

        while (true) {

            System.out.println("""

                1. Routes
                2. Fees
                3. Register
                4. Update Info
                5. Profile
                6. Pay Fee
                7. Delete
                8. Exit

            """);
```

```
int choice = input.nextInt();

switch (choice) {

    case 1 -> routeService.displayRoutes();

    case 2 -> routeService.displayFees();

    case 3 -> {
        Bus bus = busService.register(regNo);
        fileService.save(bus);
        System.out.println("====Registered Successfully!====\n");
    }

    case 4 -> {
        Bus bus = busService.update(regNo);
        fileService.save(bus);
        System.out.println("====Updated Successfully!====\n");
    }

    case 5 -> {
        if (feePaid)
            fileService.read(regNo);
        else
            System.out.println("====Pay fee first!====\n");
    }

    case 6 -> {
        System.out.println("1.JazzCash\n 2.HBL\n 3.Alfalah\n");
        int option = input.nextInt();
        paymentService.pay(option);
        feePaid = true;
    }
}
```

```

        }

        case 7 -> fileService.delete(regNo);

        case 8 -> { return; }

    }

}

}

}

```

BUS:

```

public class Bus {

    private String registrationNo;
    private String busId;
    private String route;
    private String stop;
    private String fee;
    private String pickupTime;
    private String dropTime;

    public Bus(String registrationNo, String busId,
               String route, String stop,
               String fee, String pickupTime,
               String dropTime) {

        this.registrationNo = registrationNo;
        this.busId = busId;
        this.route = route;
        this.stop = stop;
        this.fee = fee;
    }
}

```

```

        this.pickupTime = pickupTime;

        this.dropTime = dropTime;
    }

    public String getRegistrationNo() {
        return registrationNo;
    }

    public String getDetails() {
        return ""
            +
            "-----"
            +
            "Registration No : %s"
            +
            "Bus ID          : %s"
            +
            "Route            : %s"
            +
            "Stop             : %s"
            +
            "Fee              : %s"
            +
            "Pickup Time       : %s"
            +
            "Drop Time        : %s"
            +
            "-----"
            +
            """".formatted(
                registrationNo, busId, route,
                stop, fee, pickupTime, dropTime
            );
    }
}

```

BusService:

```

import java.util.Random;

import java.util.Scanner;

public class BusService {

```

```

private RouteService routeService;

private TimeService timeService = new TimeService();

private Scanner input = new Scanner(System.in);

public BusService(RouteService routeService) {
    this.routeService = routeService;
}

public Bus register(String regNo) {

    routeService.displayRoutes();

    String route;
    while (true) {
        System.out.print("Select Route: ");
        route = input.next();
        if (routeService.isValidRoute(route)) break;
        System.out.println("=====Invalid route!=====\\n");
    }

    System.out.println("Available Stops: " + routeService.getStops(route));
    input.nextLine();

    System.out.print("Enter Stop: ");
    String stop = input.nextLine();

    String fee = routeService.getFee(route);

    System.out.println("Pickup Times:");
    timeService.displayPickupTimes();
    int pickupChoice = input.nextInt();
    String pickup = timeService.getPickup(pickupChoice);

```

```

        System.out.println("Drop Times:");
        timeService.displayDropTimes();
        int dropChoice = input.nextInt();
        String drop = timeService.getDrop(dropChoice);

        String busId = String.valueOf(new Random().nextInt(9000) + 1000);

        return new Bus(regNo, busId, route, stop, fee, pickup, drop);
    }

    public Bus update(String regNo) {
        System.out.println("=====Update Information=====\\n");
        return register(regNo);
    }
}

```

RouteService:

```

import java.util.*;

public class RouteService {

    private final Map<String, List<String>> routes = new HashMap<>();
    private final Map<String, String> fees = new HashMap<>();

    public RouteService() {

        routes.put("1", Arrays.asList("F-11", "Golra More", "Margalla Road"));
        routes.put("2", Arrays.asList("G-11", "G-10", "G-9"));
        routes.put("3", Arrays.asList("Askari XI", "Qasim Market"));
        routes.put("4", Arrays.asList("Bahria Town", "PWD", "Saddar"));
    }
}

```

```

        fees.put("1", "20000");
        fees.put("2", "15000");
        fees.put("3", "14000");
        fees.put("4", "18000");
    }

    public void displayRoutes() {
        routes.forEach((k, v) ->
            System.out.println("Route " + k + " -> " + v));
    }

    public void displayFees() {
        fees.forEach((k, v) ->
            System.out.println("Route " + k + " Fee: " + v));
    }

    public boolean isValidRoute(String route) {
        return routes.containsKey(route);
    }

    public List<String> getStops(String route) {
        return routes.get(route);
    }

    public String getFee(String route) {
        return fees.get(route);
    }
}

```

TimeService:

```
import java.util.Arrays;
import java.util.List;

public class TimeService {

    private final List<String> pickupTimes =
        Arrays.asList("8 AM", "10 AM", "12 PM");

    private final List<String> dropTimes =
        Arrays.asList("1:30 PM", "2:30 PM", "5:30 PM");

    public void displayPickupTimes() {
        for (int i = 0; i < pickupTimes.size(); i++)
            System.out.println((i + 1) + ". " + pickupTimes.get(i));
    }

    public void displayDropTimes() {
        for (int i = 0; i < dropTimes.size(); i++)
            System.out.println((i + 1) + ". " + dropTimes.get(i));
    }

    public String getPickup(int index) {
        return pickupTimes.get(index - 1);
    }

    public String getDrop(int index) {
        return dropTimes.get(index - 1);
    }
}
```

PaymentService:

```
import java.awt.Desktop;

import java.net.URI;

public class PaymentService {

    public void pay(int option) {

        try {

            Desktop desk = Desktop.getDesktop();

            switch (option) {

                case 1 -> desk.browse(new URI("https://www.jazzcash.com.pk"));

                case 2 -> desk.browse(new URI("https://www.hblibank.com.pk"));

                case 3 -> desk.browse(new
URI("https://netbanking.bankalfalah.com/"));

                default -> System.out.println("=====Invalid
option=====\\n");

            }

            System.out.println("=====Payment Done!=====\\n");

        } catch (Exception e) {

            System.out.println("=====Payment Error!=====\\n");

        }

    }

}
```

Output:

Transport Management System

1. Visit transport management system
2. Previous Menu
3. Exit

Enter your choice : 1

1. Routes
2. Fees
3. Register
4. Update Info
5. Profile
6. Pay Fee
7. Delete
8. Exit

3

Route 1 -> [F-11, Golra More, Margalla Road]

Route 2 -> [G-11, G-10, G-9]

Route 3 -> [Askari XI, Qasim Market]

Route 4 -> [Bahria Town, PWD, Saddar]

Select Route: 2

Available Stops: [G-11, G-10, G-9]

Enter Stop: G-9

Pickup Times:

1. 8 AM
2. 10 AM
3. 12 PM

1

Drop Times:

1. 1:30 PM
2. 2:30 PM
3. 5:30 PM

1

====Registered Successfully!====

5

====Pay fee first!====

6

- 1.JazzCash
- 2.HBL
- 3.Alfalah

1

=====Payment Done!=====

5

Registration No : SP21-BCS-108
Bus ID : 4233
Route : 2
Stop : G-9
Fee : 15000
Pickup Time : 8 AM
Drop Time : 1:30 PM

4

=====Update Information=====

Route 1 -> [F-11, Golra More, Margalla Road]
Route 2 -> [G-11, G-10, G-9]
Route 3 -> [Askari XI, Qasim Market]
Route 4 -> [Bahria Town, PWD, Saddar]
Select Route: 2

Available Stops: [G-11, G-10, G-9]

Enter Stop: G-9

Pickup Times:

1. 8 AM
2. 10 AM
3. 12 PM

2

Drop Times:

1. 1:30 PM
2. 2:30 PM
3. 5:30 PM

3

====Updated Successfully!=====

5

```
-----  
Registration No : SP21-BCS-108  
Bus ID       : 9087  
Route        : 2  
Stop         : G-9  
Fee          : 15000  
Pickup Time  : 10 AM  
Drop Time    : 5:30 PM  
-----
```

7

Deleted Successfully

1. Routes
2. Fees
3. Register
4. Update Info
5. Profile
6. Pay Fee
7. Delete
8. Exit

5

Read error!

SOLID Application:

SOLID	Applied
SRP	YES
OCP	YES
LSP	NEUTRAL
ISP	NO
DIP	PARTIAL

Applying Code Smell

1.TransportManagement.java

```
import java.util.*;
public class TransportManagement{

    static Scanner input = new Scanner(System.in);
    static Main mainClass = new Main();    // main class object
    static TMS transportObj = new TMS();

    static void manage(String regNumber){

System.out.println("\n*****\n");

        System.out.println("\tTransport Management System\n");

System.out.println("*****\n");
;

        System.out.print("1. Visit transport management system\n");
System.out.print("2. Previous Menu\n");
System.out.print("3. Exit\n");
System.out.println();
        while(true){
            try{
                System.out.print("Enter your choice : ");
                int c = input.nextInt();
                boolean valid = false;

                switch(c){
                    case 1 : {
                        transportObj.manage(regNumber);
                        valid = true;
                        break;
                    }
                    case 2 : {
                        mainClass.management(regNumber);
                        valid = true;
                        break;
                    }
                    case 3 : {
                        System.exit(0);
                    }
                }
            }
            if(valid){
                break;
            }
        }
    }
}
```



```

        return;

        case 3:
            System.exit(0);

        default:
            System.out.println("Invalid choice! Try again.");
    }

}

catch(Exception e){
    System.out.println("Invalid input! Try again.");
    input.nextLine();
}

}
}

```

Code smell	Where
Tight Coupling	Main mainClass = new Main()
Unnecessary Object Creation	static TMS transportObj
Magic Numbers	case 1,2,3
Static Overuse	static variables
Duplicate Menu Pattern	repeated while-switch menu

2. TMS.java

```

import java.util.Scanner;

public class TMS {

    private RouteService routeService = new RouteService();
    private FileService fileService = new FileService();
    private PaymentService paymentService = new PaymentService();
    private BusService busService = new BusService(routeService);

    public void manage(String regNo) {

        Scanner input = new Scanner(System.in);
        boolean feePaid = false;
    }
}

```

```
while (true) {

    System.out.println("""

        1. Routes
        2. Fees
        3. Register
        4. Update Info
        5. Profile
        6. Pay Fee
        7. Delete
        8. Exit

        """);

    int choice = input.nextInt();

    switch (choice) {

        case 1 -> routeService.displayRoutes();

        case 2 -> routeService.displayFees();

        case 3 -> {
            Bus bus = busService.register(regNo);
            fileService.save(bus);
            System.out.println("====Registered Successfully!====\n");
        }

        case 4 -> {
            Bus bus = busService.update(regNo);
            fileService.save(bus);
            System.out.println("====Updated Successfully!====\n");
        }

        case 5 -> {
            if (feePaid)
                fileService.read(regNo);
            else
                System.out.println("====Pay fee first!====\n");
        }

        case 6 -> {
            System.out.println("1.JazzCash\n 2.HBL\n 3.Alfalah\n");
            int option = input.nextInt();
            paymentService.pay(option);
            feePaid = true;
        }

    }

}
```

```

        case 7 -> fileService.delete(regNo);

        case 8 -> { return; }
    }
}
}
}

```

Refactor:

```

import java.util.Scanner;

public class TMS {

    private RouteService routeService = new RouteService();
    private FileService fileService = new FileService();
    private PaymentService paymentService = new PaymentService();
    private BusService busService = new BusService(routeService);

    private Scanner input = new Scanner(System.in);
    private boolean feePaid = false;

    private static final int ROUTES = 1;
    private static final int FEES = 2;
    private static final int REGISTER = 3;
    private static final int UPDATE = 4;
    private static final int PROFILE = 5;
    private static final int PAY_FEE = 6;
    private static final int DELETE = 7;
    private static final int EXIT = 8;

    public void manage(String regNo) {

        while (true) {

            displayMenu();
            int choice = input.nextInt();

            switch (choice) {

                case ROUTES -> routeService.displayRoutes();

                case FEES -> routeService.displayFees();

                case REGISTER -> registerBus(regNo);

                case UPDATE -> updateBus(regNo);

```

```

        case PROFILE -> viewProfile(regNo);

        case PAY_FEE -> payFee();

        case DELETE -> fileService.delete(regNo);

        case EXIT -> { return; }

        default -> System.out.println("Invalid Option!");
    }
}

private void displayMenu() {
    System.out.println("""

        1. Routes
        2. Fees
        3. Register
        4. Update Info
        5. Profile
        6. Pay Fee
        7. Delete
        8. Exit

        """);
}

private void registerBus(String regNo) {
    Bus bus = busService.register(regNo);
    fileService.save(bus);
    System.out.println("====Registered Successfully!====");
}

private void updateBus(String regNo) {
    Bus bus = busService.update(regNo);
    fileService.save(bus);
    System.out.println("====Updated Successfully!====");
}

private void viewProfile(String regNo) {
    if (feePaid)
        fileService.read(regNo);
    else
        System.out.println("====Pay fee first!====");
}

private void payFee() {

```

```

        System.out.println("""
            1. JazzCash
            2. HBL
            3. Alfalah
            """);

        int option = input.nextInt();
        paymentService.pay(option);

        feePaid = true;
    }
}

```

Code smell	Where
Long method	<code>manage()</code> method handling menu, registration, payment, profile, delete
Magic numbers	<code>case 1,2,3,4,5,6,7,8</code> in switch statement
Switch statement smell	Large <code>switch(choice)</code> controlling many operations
Duplicate code	<code>fileService.save(bus);</code> used in both register and update
Unnecessary object creation	<code>Scanner input = new Scanner(System.in);</code> inside method

Refactoring	Where
Extract method	Created <code>registerBus()</code> , <code>updateBus()</code> , <code>viewProfile()</code> , <code>payFee()</code>
Replace magic numbers	Introduced constants <code>ROUTES</code> , <code>FEES</code> , <code>REGISTER</code> , etc.
Move Object Creation	<code>Scanner</code> moved to class variable

Reduce Long Method	<code>manage()</code> now only controls program flow
Improve Readability	Added <code>displayMenu()</code> method

3. PaymentService.java

```
import java.awt.Desktop;
import java.net.URI;

public class PaymentService {

    public void pay(int option) {

        try {
            Desktop desk = Desktop.getDesktop();

            switch (option) {
                case 1 -> desk.browse(new URI("https://www.jazzcash.com.pk"));
                case 2 -> desk.browse(new URI("https://www.hbllibank.com.pk"));
                case 3 -> desk.browse(new
URI("https://netbanking.bankalfalah.com/"));
                default -> System.out.println("=====Invalid
option=====\\n");
            }

            System.out.println("=====Payment Done!=====\\n");

        } catch (Exception e) {
            System.out.println("=====Payment Error!=====\\n");
        }
    }
}
```

Refactore:

```
import java.awt.Desktop;
import java.net.URI;

public class PaymentService {

    private static final int JAZZCASH = 1;
    private static final int HBL = 2;
    private static final int ALFALAH = 3;

    private static final String JAZZCASH_URL = "https://www.jazzcash.com.pk";
```

```

private static final String HBL_URL = "https://www.hbllibank.com.pk";
private static final String ALFALAH_URL =
"https://netbanking.bankalfalah.com/";

public void pay(int option) {

    try {

        Desktop desk = Desktop.getDesktop();

        switch (option) {

            case JAZZCASH -> desk.browse(new URI(JAZZCASH_URL));

            case HBL -> desk.browse(new URI(HBL_URL));

            case ALFALAH -> desk.browse(new URI(ALFALAH_URL));

            default -> {
                System.out.println("====Invalid option====\n");
                return;
            }
        }

        System.out.println("====Payment Done!====\n");

    } catch (Exception e) {
        System.out.println("====Payment Error!====\n");
    }
}
}

```

Code smells	Where
Magic Numbers	case 1, case 2, case 3
Hardcoded Data	"https://www.jazzcash.com.pk" , "https://www.hbllibank.com.pk" "https://netbanking.bankalfalah.com/" Inside method

Refactoring	Where
Introduce Constant	case JAZZCASH, case HBL, case ALFALAH
Replace Literal with Constant	private static final String JAZZCASH_URL, private static final String HBL_URL, private static final String ALFALAH_URL

Applying Design Pattern

1. TMS.java (Factory Pattern)

Previous:

```
import java.util.*;

public class TMS {

    private RouteService routeService = new RouteService();
    private FileService fileService = new FileService();
    private PaymentService paymentService = new PaymentService();
    private BusService busService = new BusService(routeService);

    private Scanner input = new Scanner(System.in);
    private boolean feePaid = false;

    private Map<Integer, MenuAction> actions = new HashMap<>();

    public TMS() {

        actions.put(1, new RoutesAction());
        actions.put(2, new FeesAction());
    }
}
```

```
actions.put(3, new RegisterAction());
actions.put(4, new UpdateAction());
actions.put(5, new ProfileAction());
actions.put(6, new PayFeeAction());
actions.put(7, new DeleteAction());
actions.put(8, new ExitAction());

}

public void manage(String regNo) {

    while (true) {

        displayMenu();

        int choice = input.nextInt();

        MenuAction action = actions.get(choice);

        if (action != null) {
            action.execute(regNo);
        } else {
            System.out.println("Invalid Option!");
        }

    }

}

private void displayMenu() {
```

```
System.out.println("""

    1. Routes
    2. Fees
    3. Register
    4. Update Info
    5. Profile
    6. Pay Fee
    7. Delete
    8. Exit

    """);
}

// ===== Interface =====

interface MenuAction {
    void execute(String regNo);
}

// ===== Actions =====

class RoutesAction implements MenuAction {
    public void execute(String regNo) {
        routeService.displayRoutes();
    }
}
```

```
class FeesAction implements MenuAction {  
    public void execute(String regNo) {  
        routeService.displayFees();  
    }  
}
```

```
class RegisterAction implements MenuAction {  
    public void execute(String regNo) {  
  
        Bus bus = busService.register(regNo);  
        fileService.save(bus);  
  
        System.out.println("====Registered Successfully!====");  
    }  
}
```

```
class UpdateAction implements MenuAction {  
    public void execute(String regNo) {  
  
        Bus bus = busService.update(regNo);  
        fileService.save(bus);  
  
        System.out.println("====Updated Successfully!====");  
    }  
}
```

```
class ProfileAction implements MenuAction {  
    public void execute(String regNo) {
```

```
        if (feePaid)
            fileService.read(regNo);
        else
            System.out.println("====Pay fee first!====");
    }
}
```

```
class PayFeeAction implements MenuAction {
```

```
    public void execute(String regNo) {

        System.out.println("""
            1. JazzCash
            2. HBL
            3. Alfalah
            """);

        int option = input.nextInt();

        paymentService.pay(option);

        feePaid = true;
    }
}
```

```
class DeleteAction implements MenuAction {
    public void execute(String regNo) {
        fileService.delete(regNo);
    }
}
```

```

    }
}

class ExitAction implements MenuAction {
    public void execute(String regNo) {
        System.out.println("Exiting Transport Management...");
        return;
    }
}
}

```

New:

```

import java.util.*;
import java.util.function.Supplier;

public class TMS {

    private RouteService routeService = new RouteService();
    private FileService fileService = new FileService();
    private PaymentService paymentService = new PaymentService();
    private BusService busService = new BusService(routeService);

    private Scanner input = new Scanner(System.in);
    private boolean feePaid = false;

    private MenuActionFactory actionFactory = new MenuActionFactory();

    public void manage(String regNo) {
        while (true) {
            displayMenu();
            int choice = input.nextInt();

            MenuAction action = actionFactory.createAction(choice);
            if (action != null) {

```

```

        action.execute(regNo);
        if (action instanceof ExitAction) {
            break;
        }
    } else {
        System.out.println("Invalid Option!");
    }
}
}

```

```

private void displayMenu() {
    System.out.println("

```

1. Routes
2. Fees
3. Register
4. Update Info
5. Profile
6. Pay Fee
7. Delete
8. Exit

```

    """);
}

```

```

// ===== Interface =====

```

```

interface MenuAction {
    void execute(String regNo);
}

```

```

// ===== Factory without switch =====

```

```

class MenuActionFactory {

```

```

    private final Map<Integer, Supplier<MenuAction>> actionMap = new HashMap<>();

```

```

    public MenuActionFactory() {
        actionMap.put(1, RoutesAction::new);
        actionMap.put(2, FeesAction::new);
        actionMap.put(3, RegisterAction::new);
        actionMap.put(4, UpdateAction::new);
    }
}

```

```

        actionMap.put(5, ProfileAction::new);
        actionMap.put(6, PayFeeAction::new);
        actionMap.put(7, DeleteAction::new);
        actionMap.put(8, ExitAction::new);
    }

    public MenuAction createAction(int choice) {
        Supplier<MenuAction> actionSupplier = actionMap.get(choice);
        return actionSupplier != null ? actionSupplier.get() : null;
    }
}

// ===== Actions =====

class RoutesAction implements MenuAction {
    public void execute(String regNo) {
        routeService.displayRoutes();
    }
}

class FeesAction implements MenuAction {
    public void execute(String regNo) {
        routeService.displayFees();
    }
}

class RegisterAction implements MenuAction {
    public void execute(String regNo) {
        Bus bus = busService.register(regNo);
        fileService.save(bus);
        System.out.println("=====Registered Successfully!=====");
    }
}

class UpdateAction implements MenuAction {
    public void execute(String regNo) {
        Bus bus = busService.update(regNo);
        fileService.save(bus);
        System.out.println("=====Updated Successfully!=====");
    }
}

```

```

    }

    class ProfileAction implements MenuAction {
        public void execute(String regNo) {
            if (feePaid)
                fileService.read(regNo);
            else
                System.out.println("====Pay fee first!====");
        }
    }

    class PayFeeAction implements MenuAction {
        public void execute(String regNo) {
            System.out.println("""
                1. JazzCash
                2. HBL
                3. Alfalah
                """);
            int option = input.nextInt();
            paymentService.pay(option);
            feePaid = true;
        }
    }

    class DeleteAction implements MenuAction {
        public void execute(String regNo) {
            fileService.delete(regNo);
        }
    }

    class ExitAction implements MenuAction {
        public void execute(String regNo) {
            System.out.println("Exiting Transport Management...");
        }
    }
}

```

Benefits of Using Factory Here

1. **Decouples creation from usage**

- **TMS** no longer needs to know the concrete classes.
- 2. **Easier to add new actions**
 - Just add a new class and update the factory.
- 3. **Single responsibility**
 - Factory handles creation, **TMS** handles management.
- 4. **Open/Closed Principle**
 - Adding new features doesn't require changing existing logic much.

Architectural Pattern and Diagram:

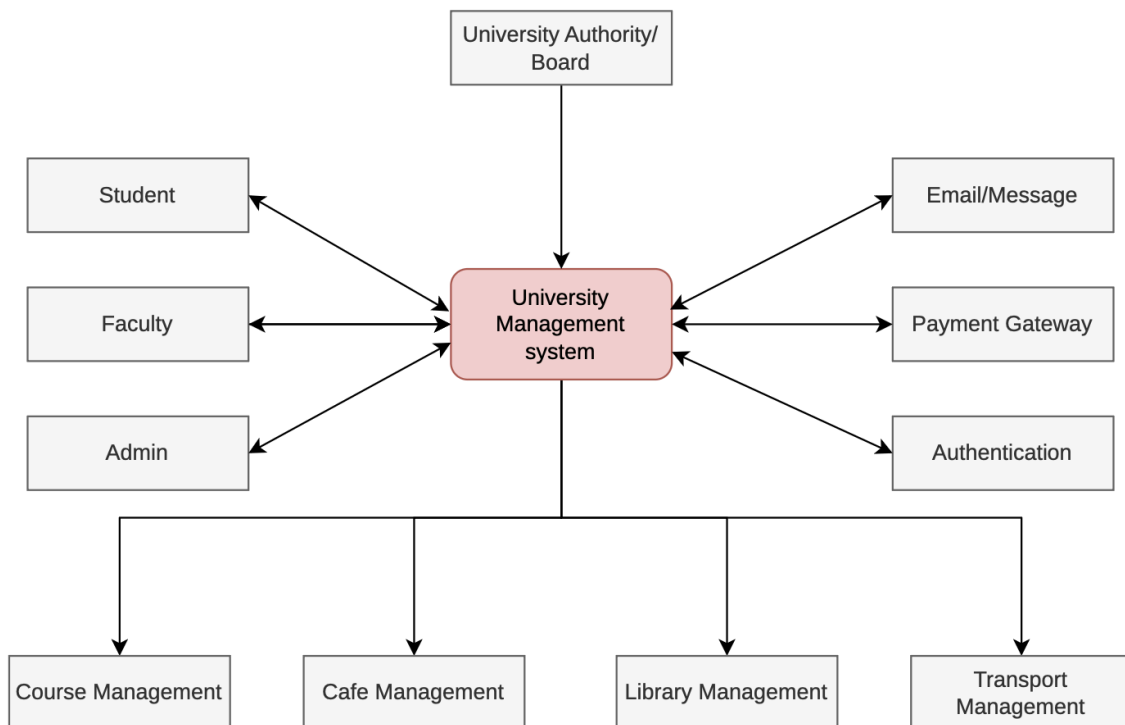


Figure: Context Diagram.

Here,

1. Users:

- **Student to UMS system:** Registration info, course, enrollment, fee payment, and requests.
- **UMS system to Student:** Results, attendance, profile info, notices, etc.
- **Faculty to UMS system:** Attendance, marks, materials, and feedback.
- **UMS system to Faculty:** Student list, schedule, course info, reports, notices.
- **Admin to UMS system:** Reports, logs, statistics, alerts, confirmations.
- **UMS system to Admin:** Data entry, updates, configurations, requests.

2. Peers:

- **Payment Gateway:** process payment, transaction status.
- **Email:** Send emails/SMS.
- **Authentication:** User login, identity verification

3. Superordinate: University authority

4. Subordinate: Course management, cafe management, transport management, library management.

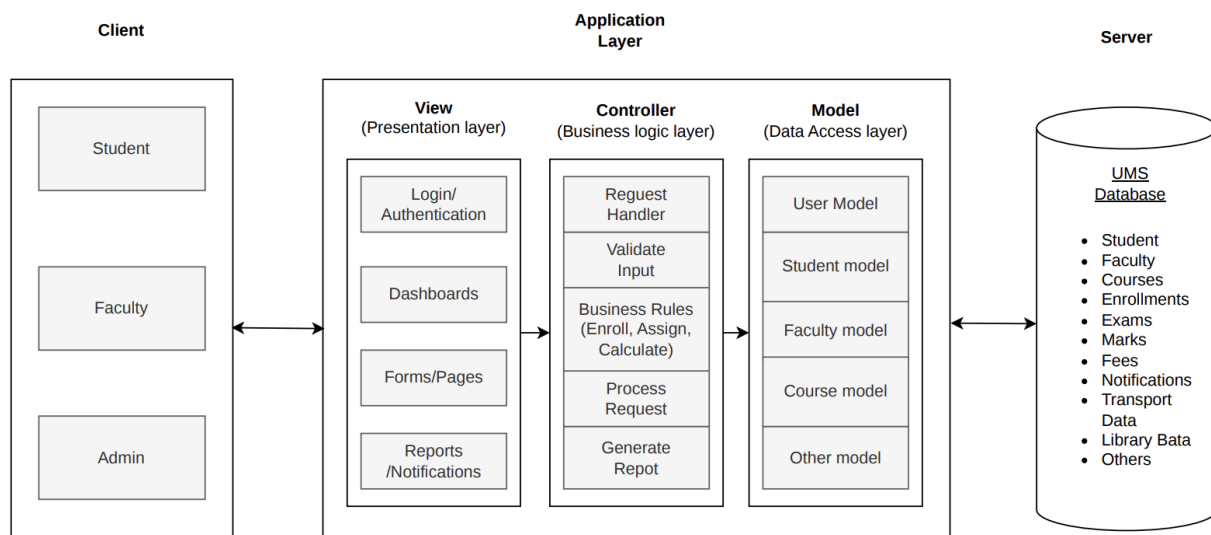


Figure: Client-server Architecture using MVC Approach

In the future, we can develop our system's architecture by using Client-Server and MVC architectural patterns, which then work seamlessly and perfectly.

